CareFlow HMS

DBMS Experiential Learning Level 3 Project Report

Project title for the lab sheet careflow-hms
Database option Option 2, PostgreSQL and pgvector
Revision 6 November 2026
Repository https://github.com/Catherine61005/careflow-hms

Team member	SRN	Presentation workstream
Catherine D	PES1UG24CS120	Relational SQL, data integrity, and the scheduling and inpatient workflows
Bhavesh Velluru	PES1UG24CS115	Vector database, embedding lifecycle, Smart Triage, and Similar Past Cases

Problem and objectives

A hospital front desk must route free-text patient complaints to the right department, avoid overlapping bookings, and keep room allocations and bills consistent as staff work concurrently.

• Model hospital operations and supporting data in PostgreSQL using keys, relationships and integrity constraints.

• Execute admission, room transfer, discharge and billing workflows with database transactions and locking.

• Retrieve meaningfully similar clinical text using embeddings and use the matches to suggest a department and appointment.

• Provide a role-based Next.js application for scheduling, care records, billing and operational reporting.

Report contents

Functional requirements and architecture / ER and relational design / Data dictionary / SQL and vector implementation / Important queries / Application screenshots / Tests, contributions and conclusion

Application design

Workflow	Functional requirement	Primary access
Accounts	Sign in and sign out. Manage staff roles, disable accounts and inspect the audit trail.	Staff login; admin manages
Patient records	Register, edit and search by name, phone, ID or email. View clinical and visit history.	Admin, receptionist, doctor
Appointments	Book and cancel visits. Reject past and overlapping bookings. Record completion or no-show.	Front desk; doctor updates
Smart Triage	Use free text to recommend departments, doctors and slots. Compare keyword and semantic results.	Signed-in staff
Inpatient care	Admit, transfer and discharge patients. Track room occupancy and retrieve similar completed cases.	Front desk; staff view
Billing	Generate discharge bill, accept payments, reject overpayment and export filtered bills.	Admin and receptionist
Clinical and reports	Order tests, record abnormal results, issue prescriptions and view dashboards or revenue reports.	Doctor for clinical actions; role-based reporting

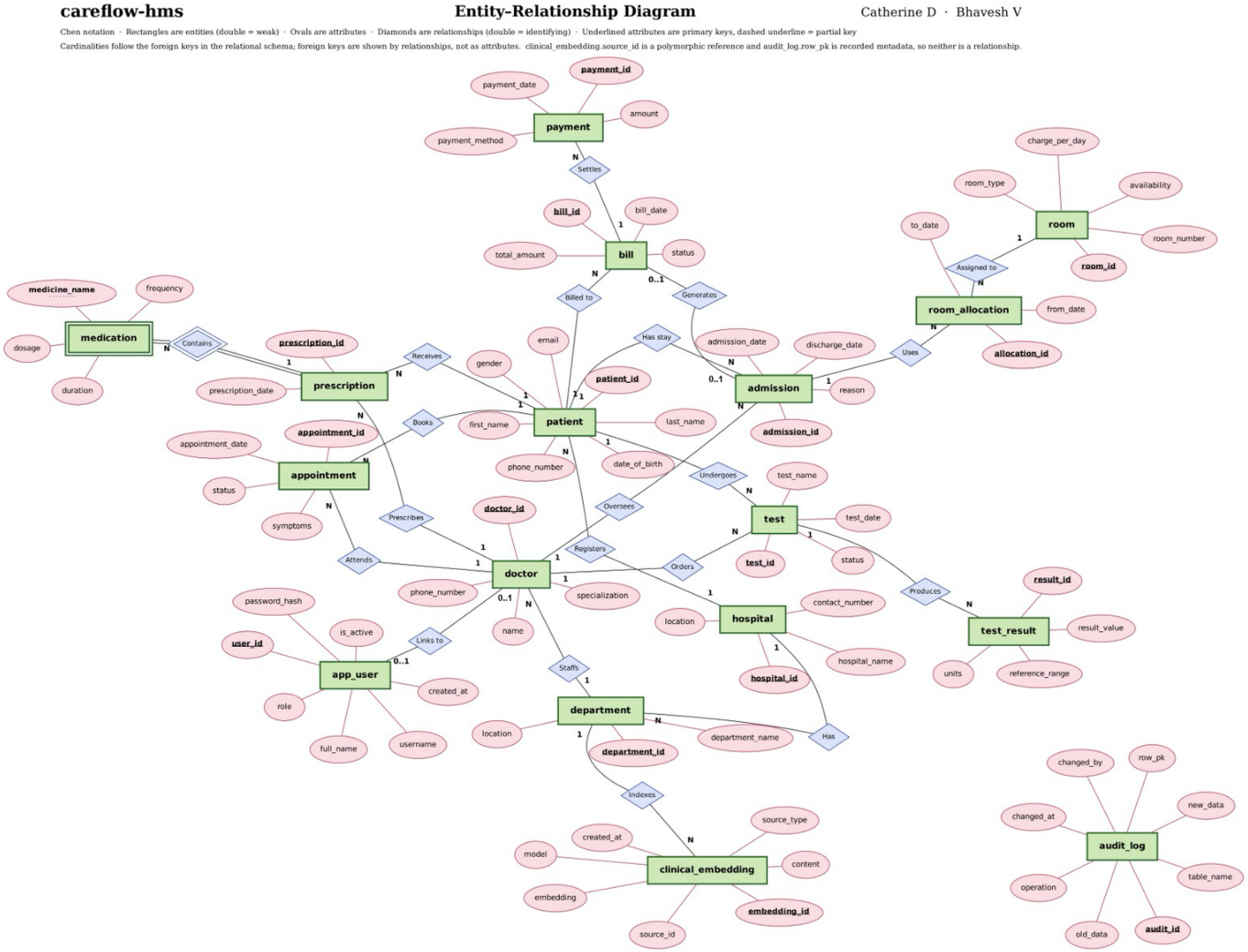
Request path

Browser (Next.js App Router, React, TypeScript) > **Route Handlers** (Zod, session and role checks) > **pg pool** (bound SQL parameters) > **PostgreSQL 16** (relational tables and pgvector).

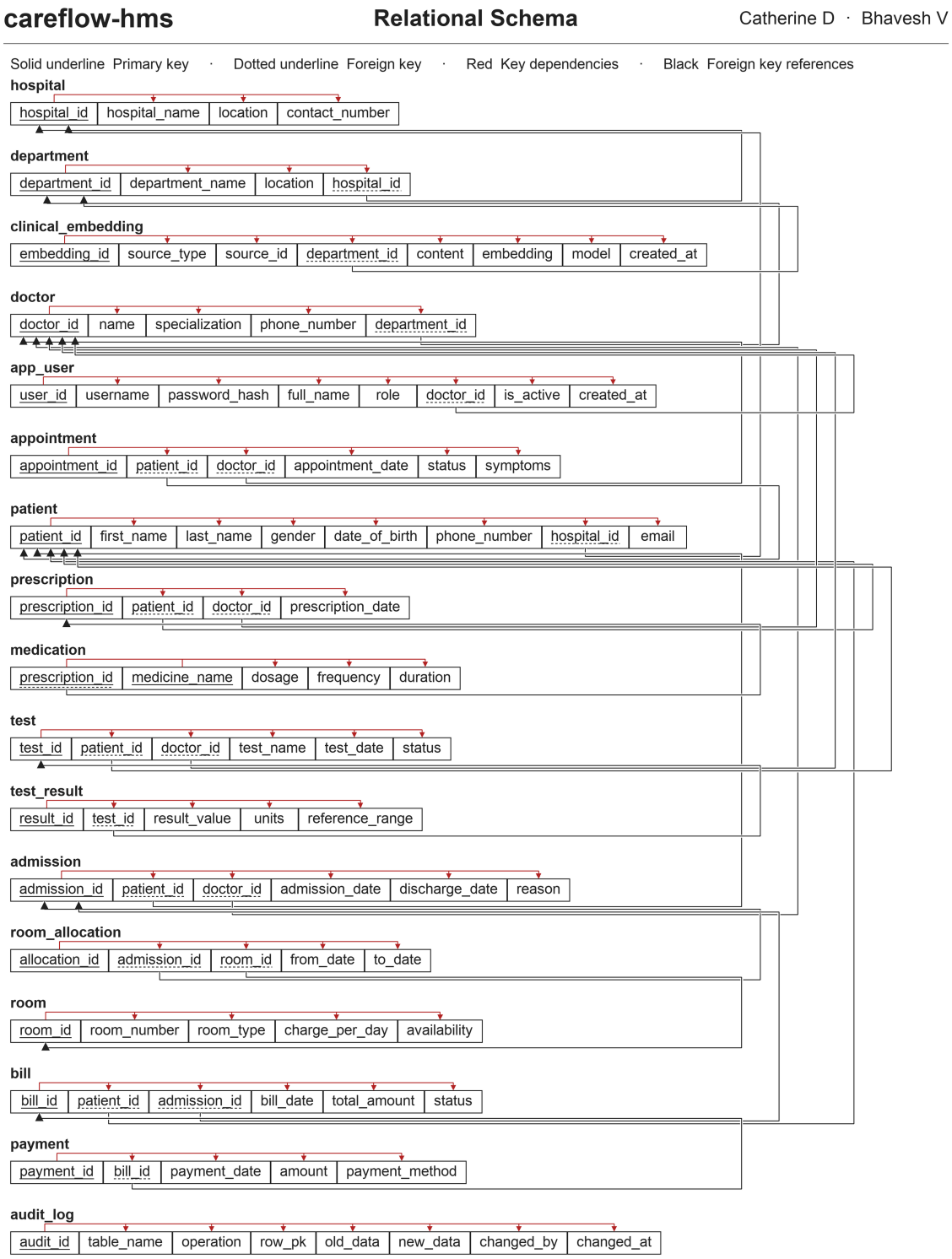
Login uses bcrypt and a signed JWT in an HTTP-only cookie. Protected routes recheck the account in the database. Connection settings and the JWT secret are read from environment variables. The application connects with the hms_app database role, which lacks schema ownership.

For semantic requests, the server loads MiniLM through Transformers.js, embeds the complaint locally, and queries the vector index and relational metadata in the same database.

Entity-relationship diagram



Relational schema



Hospital and scheduling

From the implemented PostgreSQL schema after migrations. PK = primary key, FK = foreign key, UQ = unique constraint.

hospital

Facility master record.

hospital_id integer, PK • **hospital_name** character varying(120) • **location** character varying(120) • **contact_number** phone_no

department

Named clinical unit within a hospital.

department_id integer, PK • **department_name** character varying(80) • **location** character varying(100) • **hospital_id** integer, FK to hospital

doctor

Clinician in a department.

doctor_id integer, PK • **name** character varying(100) • **specialization** character varying(80) • **phone_number** phone_no, UQ • **department_id** integer, FK to department

patient

Registration and contact details.

patient_id integer, PK • **first_name** character varying(60) • **last_name** character varying(60) • **gender** character varying(10) • **date_of_birth** date • **phone_number** phone_no • **hospital_id** integer, FK to hospital • **email** character varying(120), nullable

appointment

Doctor and patient booking, visit status and optional complaint.

appointment_id integer, PK • **patient_id** integer, FK to patient • **doctor_id** integer, FK to doctor • **appointment_date** timestamp without time zone • **status** character varying(12) • **symptoms** text, nullable

Inpatient and billing

From the implemented PostgreSQL schema after migrations. PK = primary key, FK = foreign key, UQ = unique constraint.

admission

Inpatient episode with admission and optional discharge dates.

admission_id integer, PK • **patient_id** integer, FK to patient • **doctor_id** integer, FK to doctor • **admission_date** date • **discharge_date** date, nullable • **reason** text

room

Ward inventory, tariff and operational status.

room_id integer, PK • **room_number** character varying(10), UQ • **room_type** character varying(15) • **charge_per_day** money_inr • **availability** character varying(12)

room_allocation

Dated room assignment during an admission.

allocation_id integer, PK • **admission_id** integer, FK to admission • **room_id** integer, FK to room • **from_date** date • **to_date** date, nullable

bill

Patient invoice, optionally tied to one admission.

bill_id integer, PK • **patient_id** integer, FK to patient • **admission_id** integer, nullable, FK to admission / UQ • **bill_date** date • **total_amount** money_inr • **status** character varying(15)

payment

Individual payment against a bill.

payment_id integer, PK • **bill_id** integer, FK to bill • **payment_date** date • **amount** money_inr • **payment_method** character varying(12)

Clinical records

From the implemented PostgreSQL schema after migrations. PK = primary key, FK = foreign key, UQ = unique constraint.

prescription

Medication order header.

prescription_id integer, PK • **patient_id** integer, FK to patient • **doctor_id** integer, FK to doctor • **prescription_date** date

medication

Medicine within a prescription. The owner key and medicine name form one composite primary key.

prescription_id integer, PK / FK to prescription • **medicine_name** character varying(100), PK • **dosage** character varying(30) • **frequency** character varying(30) • **duration** character varying(30)

test

Lab order and its status.

test_id integer, PK • **patient_id** integer, FK to patient • **doctor_id** integer, FK to doctor • **test_name** character varying(100) • **test_date** date • **status** character varying(12)

test_result

Measured lab result and reference range.

result_id integer, PK • **test_id** integer, FK to test • **result_value** character varying(40) • **units** character varying(20), nullable • **reference_range** character varying(40), nullable

Access, audit and vectors

From the implemented PostgreSQL schema after migrations. PK = primary key, FK = foreign key, UQ = unique constraint.

app_user

Credentials, role and account lifecycle.

user_id integer, PK • **username** character varying(40), UQ • **password_hash** text • **full_name** character varying(100) • **role** character varying(15) • **doctor_id** integer, nullable, FK to doctor / UQ • **is_active** boolean • **created_at** timestamp with time zone

audit_log

Change snapshots and actor metadata.

audit_id bigint, PK • **table_name** text • **operation** text • **row_pk** text, nullable • **old_data** jsonb, nullable • **new_data** jsonb, nullable • **changed_by** text • **changed_at** timestamp with time zone

clinical_embedding

Admission or department text with a 384-dimensional vector and model metadata.

embedding_id integer, PK • **source_type** character varying(12) • **source_id** integer • **department_id** integer, FK to department • **content** text • **embedding** vector(384) • **model** character varying(60) • **created_at** timestamp with time zone

The vector table has a composite unique constraint on (source_type, source_id). The source ID is polymorphic, and the audit row key is recorded metadata. Neither is a declared foreign key. The only optional one-to-one links are bill/admission and app_user/doctor.

PostgreSQL + pgvector

Why Option 2

Scheduling, admissions, billing and clinical records have stable relationships and integrity rules. PostgreSQL supplies constraints, locks, transactions and joins. Admission reasons and department descriptions are free text. pgvector retrieves related text even when the wording differs and keeps vector metadata joinable to doctors, stays and bills.

Feature	Implemented example
DDL and DML	CREATE tables, domains and indexes in db/01_schema.sql. ALTER and DROP demonstrations in db/05_alter_migration.sql. Patient APIs illustrate INSERT, SELECT, UPDATE and admin-only DELETE.
Transactions and locks	The API runs multi-step writes in a transaction. Admission locks the room and payment validation locks the bill.
Functions and procedure	fn_admit_patient, fn_transfer_room and sp_discharge_patient implement room lifecycle and billing.
Triggers and constraints	EXCLUDE guards booking slots and room periods. CHECK, FK and UNIQUE validate data. Triggers enforce payment limits and update room and bill state.
Views and indexing	Patient, occupancy and bill views support UI reads. A materialized view supports revenue. GIN supports full-text and trigram searches.
Vector similarity	HNSW indexes cosine distance. Retrieval compares vectors from the same model and joins results to relational data.

Embedding and retrieval lifecycle

Xenova/all-MiniLM-L6-v2 creates mean-pooled, normalized 384-dimensional vectors. Backfill refreshes admission vectors when model, source content or department metadata changes. New admissions are indexed after a successful transaction and response. Failed model loads can be retried by a later request.

Smart Triage uses the nearest 20 candidates to weight department recommendations, then uses SQL to find doctors and free slots. Similar Past Cases retrieves completed admissions in the same vector space, excludes the source admission and joins outcomes and bill amounts.

SQL and vector retrieval

Query	What it demonstrates
Q1, Q2	Multi-table JOIN for admission history; GROUP BY, HAVING and average stay.
Q3–Q5	Correlated subquery, NOT EXISTS anti-join and IN subquery.
Q6–Q9	CTEs, running totals, ranking, ward occupancy and receivables ageing.
Q10–Q13	Full-text ranking, trigram similarity, pgvector cosine retrieval and EXPLAIN for the schedule index.
Q14, Q15	Window ranking of average completed stays; same-department follow-up detection with NOT EXISTS and HAVING.

Reproducible pgvector example

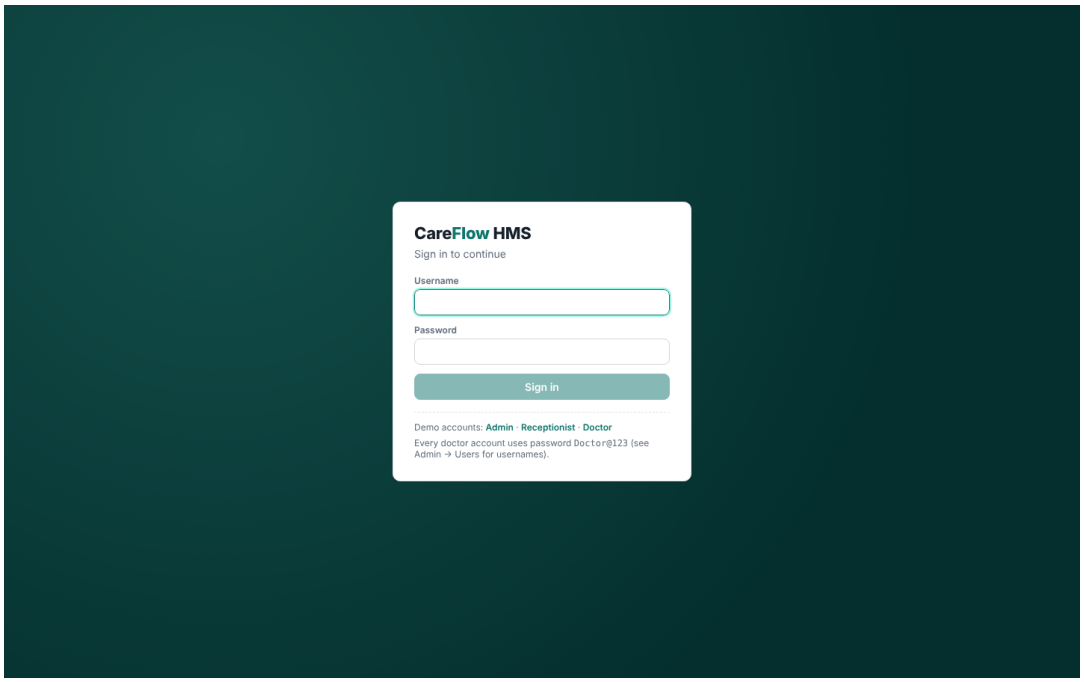
This read-only query uses one existing admission vector as the probe. Q12 in the source illustrates the parameterized variant used by the application.

```
SET search_path TO hms, public;
WITH probe AS (
  SELECT source_id, model, embedding
  FROM clinical_embedding
  WHERE source_type = 'admission'
  ORDER BY source_id LIMIT 1
)
SELECT e.source_id, e.department_id,
       1 - (e.embedding <=> p.embedding) AS similarity
FROM clinical_embedding e CROSS JOIN probe p
WHERE e.source_type = 'admission' AND e.model = p.model
  AND e.source_id <> p.source_id
ORDER BY e.embedding <=> p.embedding
LIMIT 5;
```

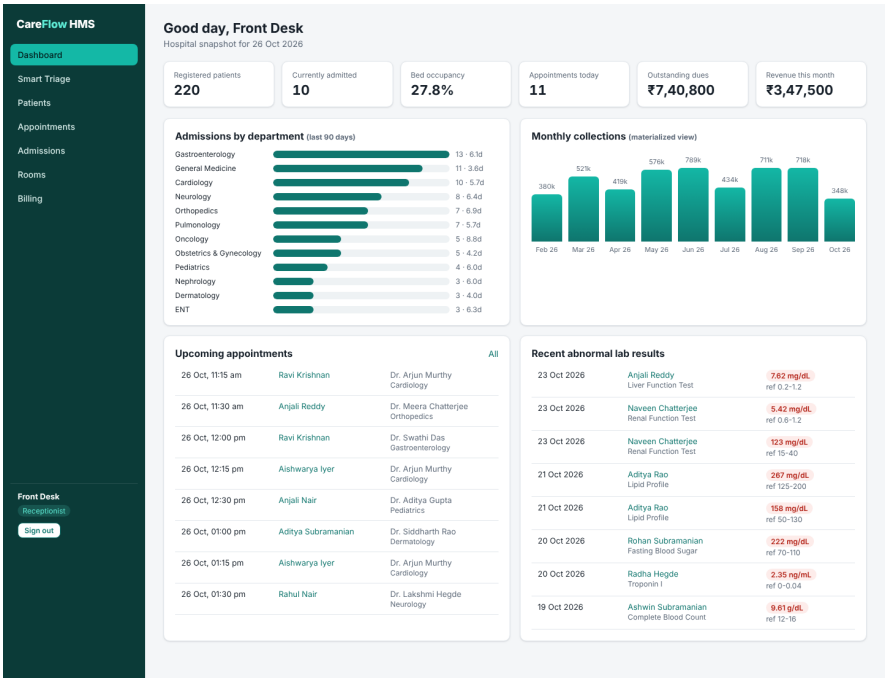
Source and output db/07_queries.sql. The submission verification also ran this stored-vector query against the seeded database.

Staff access and operations

Staff login

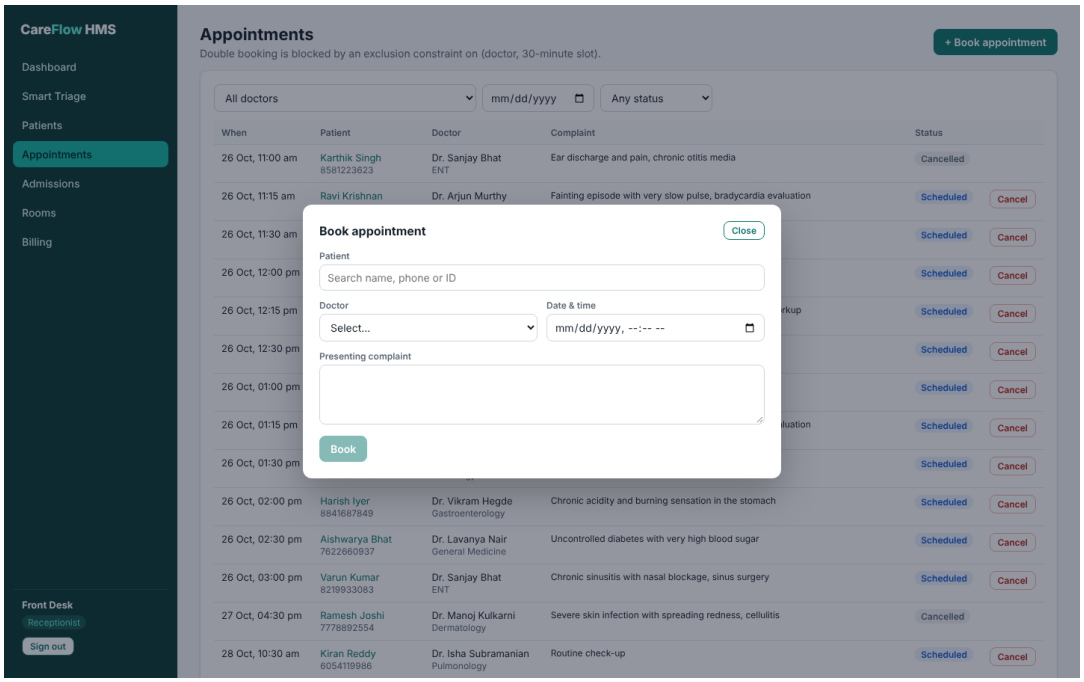


Receptionist dashboard

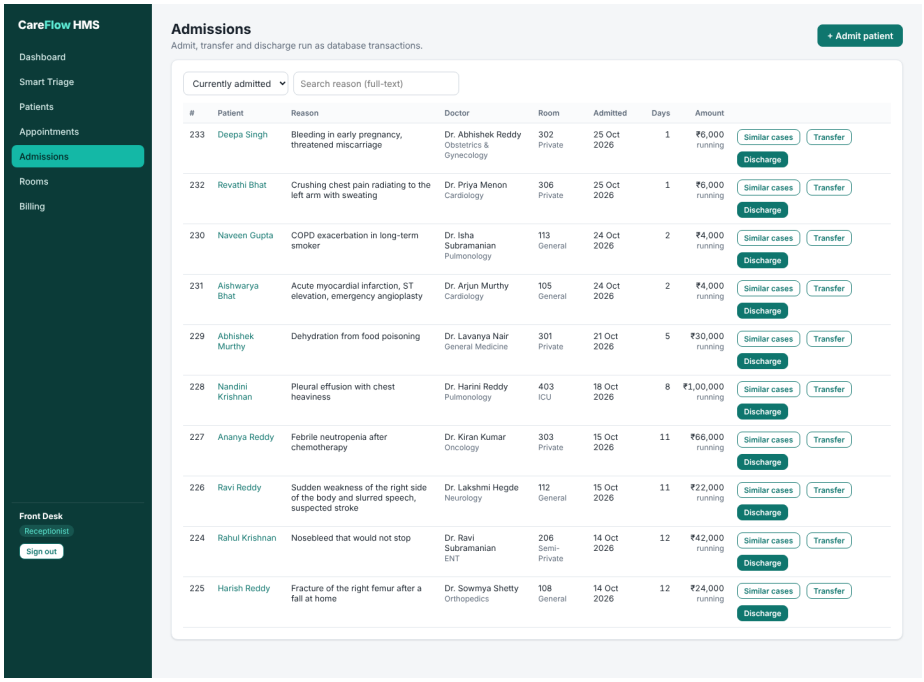


Scheduling and inpatient care

Booking a doctor and appointment slot

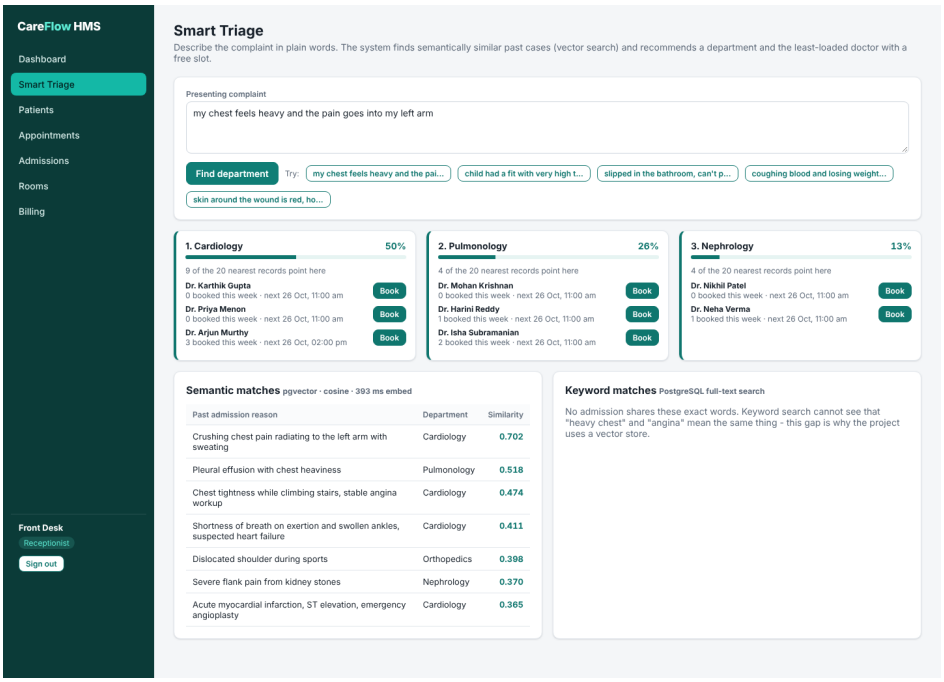


Admissions and inpatient actions

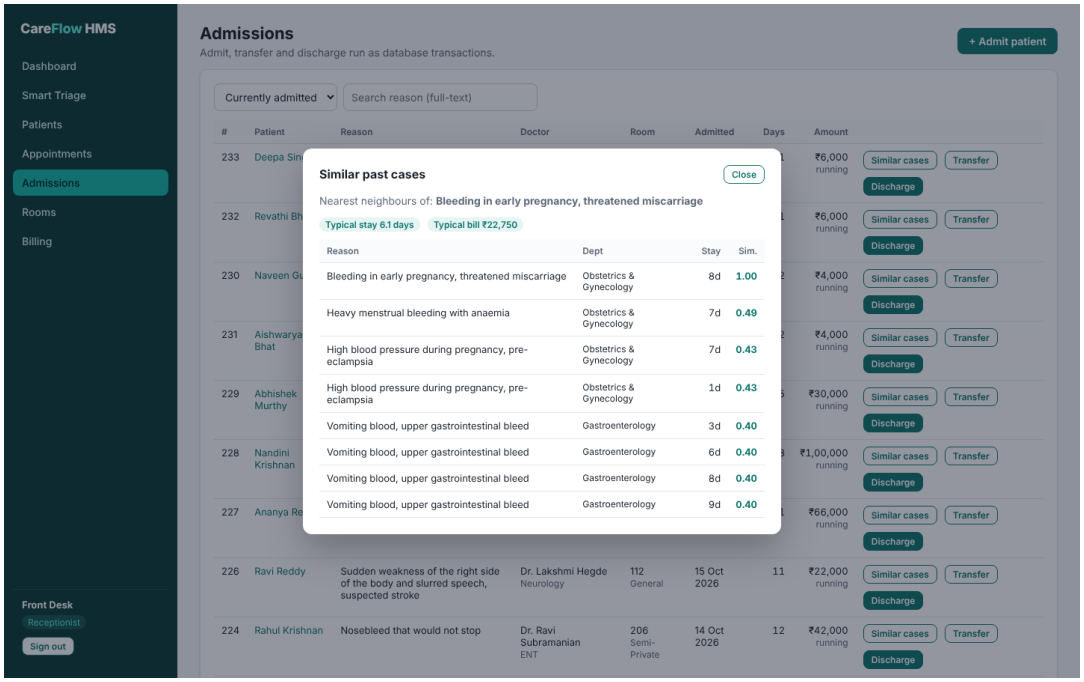


Semantic retrieval

Smart Triage, department recommendations and keyword comparison

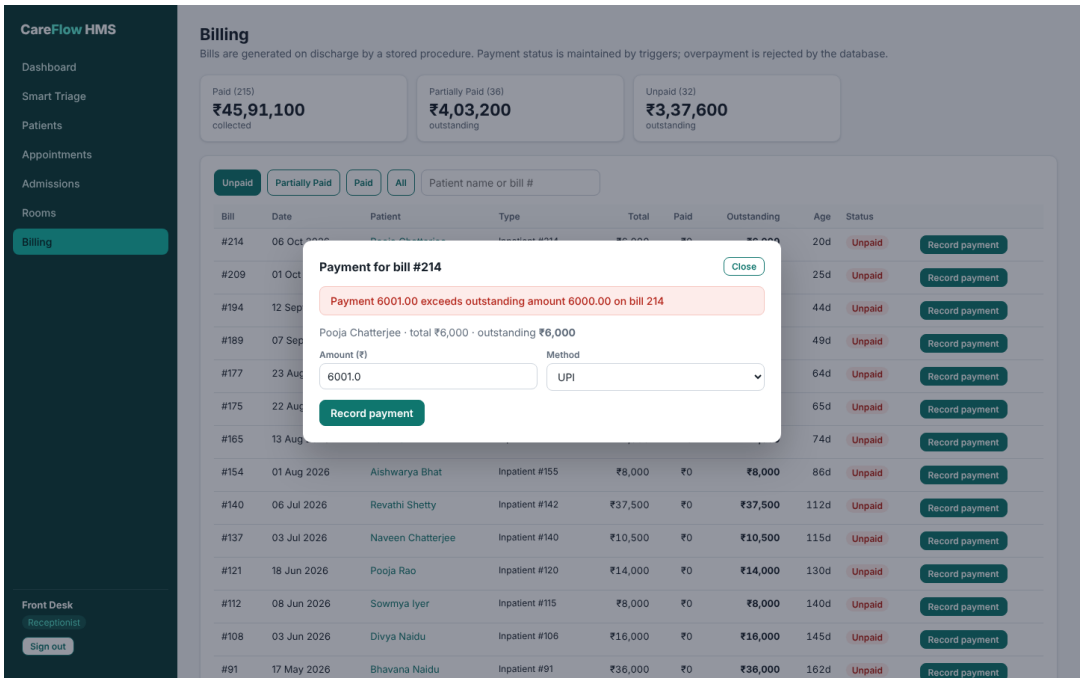


Similar Past Cases with stay and bill summaries

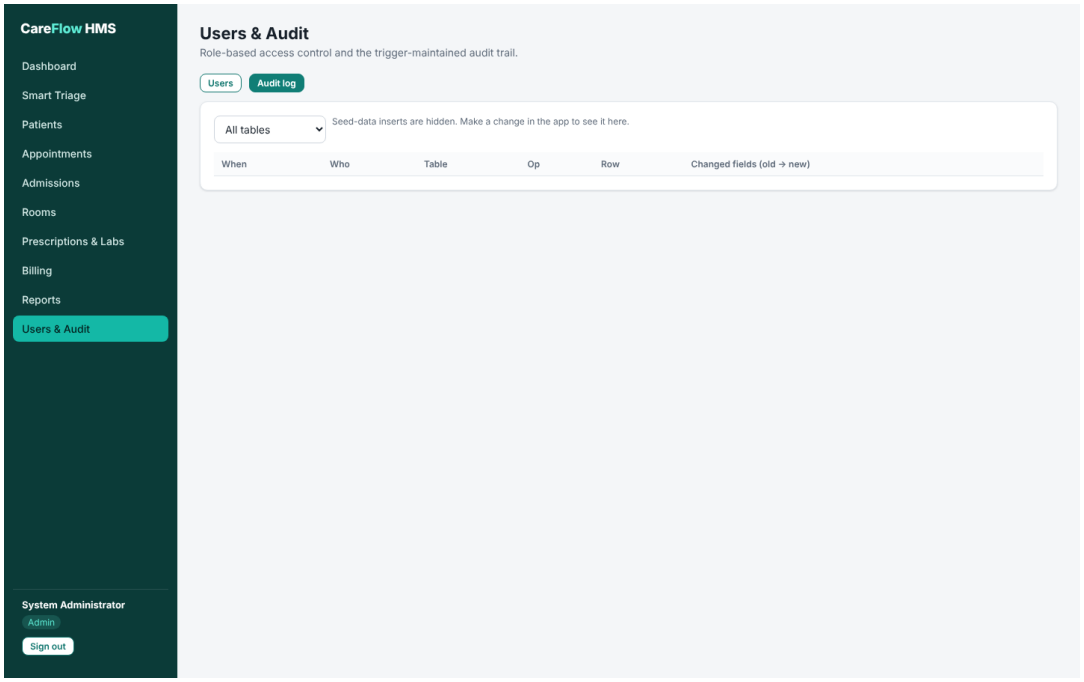


Finance and accountability

Billing rejects an overpayment



Administrative audit trail



Test results

Verification on 6 November 2026 used PostgreSQL 16.15 with pgvector in Docker and a local Next.js server. The API suite used an isolated seeded development database. The fresh Compose setup was checked separately on a new volume.

Check	Observed result
Fresh Compose initialization	Initialization completed and the database contained 220 seeded patients.
Database integrity	38 passed, 0 failed. Tests rolled back their data changes.
API end-to-end	33 passed, 0 failed. API25 and API26 exercised triage; API27 waited for and retrieved an admission embedding.
Embedding inventory	Backfill created 12 department vectors and 233 admission vectors. One additional admission vector appeared after the API suite. All measured vectors had 384 dimensions.
Important queries	The SQL query file ran with ON_ERROR_STOP. The stored-vector example returned five neighbours.
Production build	Next.js production compilation and TypeScript completed.

Representative cases

Scheduling An overlapping 30-minute slot for the same doctor is rejected.
Inpatient Transfer frees the previous room and billing uses both room tariffs.
Finance Overpayment is rejected and unauthorized CSV access returns 401 or 403.
Vector The cardiac complaint routes to Cardiology with zero keyword overlap. Similar-case results exclude the new admission itself.

Raw results are stored in docs/test-results. Both team members should repeat the smoke checks on their demonstration systems.

Team demonstration

The lab evaluates each member individually. The planned split is 12 minutes each for the demonstration, followed by three minutes of individual questions each.

Presenter	Database workstream	Next.js and verification
Catherine D	ER and schema, keys and constraints, DDL/DML, functions, triggers and SQL reporting.	Scheduling overlap and inpatient lifecycle, with SQL integrity tests and contribution diffs.
Bhavesh Velluru	Vector schema and metadata, MiniLM, backfill, cosine retrieval and HNSW.	Smart Triage and Similar Past Cases, with API25–API27 and vector contribution diffs.

GitHub evidence

The repository history records the initial implementation and subsequent improvements. The current demo assignments describe which areas each member will present.

Catherine 0cef2f6 schema and database logic, d37a5bb patient email search, 28ad2f8 missed follow-ups.
Bhavesh 9cc3446 vector retrieval and backfill, 269df9e similar-cases test, 996573b admission indexing, and PR #1 ward and billing changes.

Conclusion

CareFlow HMS connects relational integrity and semantic retrieval in one hospital workflow. PostgreSQL constraints and transactional routines maintain consistent scheduling, room allocation and billing. pgvector uses the meaning of complaints and past admissions to support routing and case retrieval. The Next.js interface exposes both workflows with role-based access.

The system is a demo using seeded hospital data and a small English-language embedding model. Triage recommendations are decision support. The model and corpus limit how well the system generalizes.

Submission

Bring the matching A4 diagram printouts and run the project on both laptops. After the live demonstration, submit this report and the GitHub repository link in the official form.